CSE 210
Computer Architecture Sessional

Assignment-1: 4-bit ALU Simulation

Section - A2
Group - 02

Members of the Group:

i 2105043 - Monjur Hossain Khan

ii 2105046 - Mohammad Raihan Rashid

iii 2105047 - Himadri Gobinda Biswas

iv 2105049 - Amit Saha

v 2105055 - Md. Abrar Jahin

1 Introduction

The **Arithmetic and Logic Unit (ALU)** serves as the computational core of a computer, responsible for performing essential mathematical and logical operations. It is fundamentally a **combinational digital circuit** that executes arithmetic and bitwise logical operations on integer binary numbers. Being a key component of computing hardware, the ALU is vital to both **Central Processing Units (CPUs)** and **Graphics Processing Units (GPUs)**. An ALU comprises two primary components: the **Arithmetic Unit** and the **Logic Unit**, which together handle a broad set of operations. These include arithmetic functions like addition, subtraction, incrementing, decrementing, and transferring, as well as logical functions such as **NOT**, **OR**, **XOR**, and **AND**. A **control unit** directs data from registers to the ALU's inputs, and **selection lines** or bits determine which operation to perform. In general, an ALU with k *selection lines* can handle 2^k distinct operations. For our implementation, the ALU is configured with *three selection inputs*. The **parallel adder** forms the core of the arithmetic section of the ALU, enabling various arithmetic operations by utilizing **multiplexed inputs**.

Logical operations, on the other hand, are carried out using specific **Integrated Circuits (ICs)**, although in optimized designs, an IC might perform operations beyond its intended use. In our design, we incorporate **four status flags** or outputs, labeled as **C (Carry Flag)**, **Z (Zero Flag)**, **V (Overflow Flag)**, and **S (Sign Flag)**. These flags convey the following:

CF: The **Carry Flag (CF)** reflects the C_{out} of the ALU's adder. If a carry is generated, this flag is set to **1**; otherwise, it remains **0**. All logical operations result in a CF value of **0**.

ZF: The **Zero Flag (ZF)** is set if the ALU's most recent operation yields a result of **0**; otherwise, it is cleared.

VF: The **Overflow Flag (VF)** is set when an arithmetic operation involving two positive numbers results in a negative outcome or when two negative numbers result in a positive outcome, indicating overflow. Logical operations reset this flag.

$$V = C_3 \oplus C_{out} \tag{1}$$

Here, C_3 represents the carry generated during the addition of A_2, B_2 & C_2, while C_{out} is the output carry from the adder.

SF: The **Sign Flag (SF)** reflects the **most significant bit (MSB)** of the ALU's output.

2 Problem Specification with Assigned Instructions

Design a 4-bit ALU with three selection bits cs0, cs1 and cs2 for performing the following operations:

Control Signals			Functions	Description
cs2	cs1	cs0		
0	0	0	Add	$A + B$
0	X	1	Sub	$A - B$
0	1	0	AND	$A \wedge B$
1	0	0	XOR	$A \oplus B$
1	X	1	Complement A	$\overline{A}$
1	1	0	NEG A	$-A$

Table 1: Problem Specification

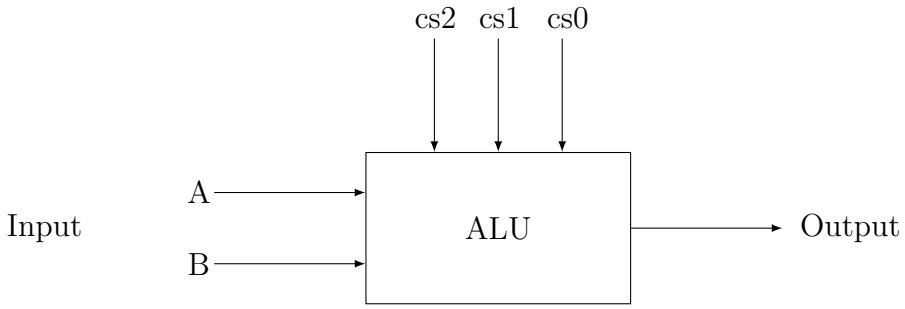


Figure 1: 4-bit ALU

3 Detailed Design Steps with K-maps

3.1 Design Steps

1. The arithmetic logical unit computes 3 arithmetic operations (Add, Sub, and, NEG A) and 3 logical operations(AND, XOR and, Complement A) with the help of two 4-bit full adder and three multiplexer.

2. For the adder, the **first input** X_i varies with different control bit input. It depends on **both** A_i **and** B_i. There are 4 possible values of X_i: A_i**,** $A \wedge B$**,** $A \oplus B$ **and** $\overline{A_i}$. Thus, we had to use ***two*** **dual 4×1 MUX** for choosing the correct required value. We also generated S_1 (k-map: 3.2.1) and S_2 (k-map: 3.2.2) for selecting the proper value for the required operation.

3. The second input Y_i also varies with different control bit input. These are as follows: B_i**,** $\overline{B_i}$ and **0**. For the selection we used ***one*** **quad 2×1 MUX**. The selection between B_i and $\overline{B_i}$ is done using S_3 (k-map:3.2.3). For generating **0** we used the another function for en_3 (k-map: 3.2.5).

4. We also generated C_{in} (k-map: 3.2.4), as the input of C_{in} varies according the different required operations and it is kept **0** during logical operations.

5. For the **Overflow Flag (VF)**, we needed both C_3 and C_{out}. Now, the tricky part was how to get the C_3 (since it is handled internally in the adder). We had a lot of design decision to make which are further discussed in the discussions. But we opted to using two adders. The first three bits of X_i and Y_i are added in the first adder. The the S_3 (C_3) from the first adder is given as C_{in} to the second adder. Now we have a C_3 and we get the required C_{out} from the S_1 terminal of the second adder. This, C_{out} is also the **Carry Flag (CF)** of the adder.

6. **Zero flag (ZF)** is computed by firstly ORing the two pairs of output bits respectively ($S_0 \vee S_1$) and ($S_2 \vee S_3$). Then we pass the output into a NOR gate and finally get the ZF as output.

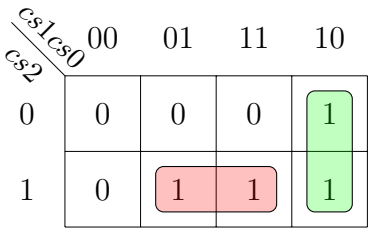
7. **The Sign Flag (SF)** is technically the MSB. So, we just take the output S_0 from the second adder as SF.

3.2 K-maps

We will be following Table 2 to construct the K-maps for intermediate selection bits.

3.2.1 K-map for S_1

S_1 is the first selection bit for the dual 4×1 Multiplexer that selects the first input of the adder. There are 4 possible values of X_i: A_i, $A \wedge B$, $A \oplus B$ **and** $\overline{A_i}$.

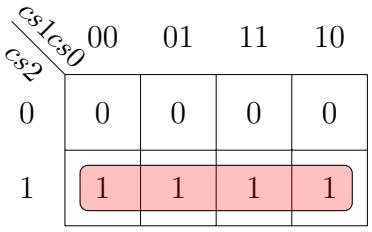


So, there are four minterms. We use basic operations (AND, OR, NOT, NOR) IC to implement it.

$$S_1 = cs2 \cdot cs0 + cs1 \cdot \overline{cs0}$$

3.2.2 K-map for S_2

S_2 is the second selection bit for the dual 4×1 Multiplexer that selects the first input of the adder.

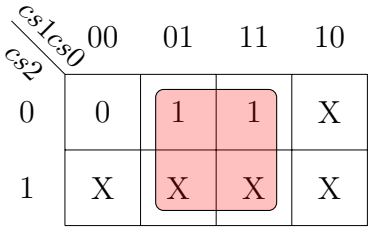


S_2 is simply $cs2$.

$$S_2 = cs2$$

3.2.3 K-map for S_3

S_3 is the selection bit for Y_i. The possible valued determined by S_3 are as follows: B_i **, and** $\overline{B_i}$.

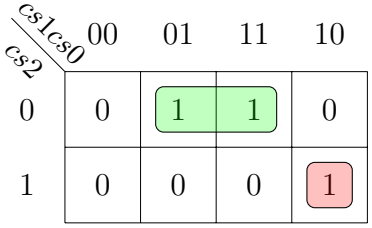


S_3 is simply $cs0$

$$S_3 = cs0$$

3.2.4 K-map for C_{in}

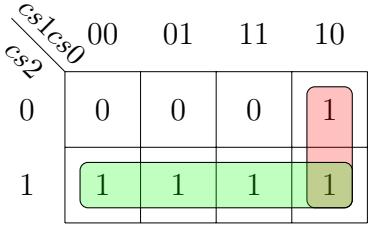
It is the input carry bit of the adder. It will be 1 at the time of *Sub and NEG A* operation.



$$C_{in} = \overline{cs2} \cdot cs0 + cs2 \cdot cs1 \cdot \overline{cs0}$$

3.2.5 K-map for en_3

en_3 for the second multiplexer is used for making all the bits of Y_i **0**. In all operations enable is high(resulting Y_i being **0**), except for add and sub operation.



The equation of en_3 is as follows:

$$c = cs2 + cs1 \cdot \overline{cs0}$$

4 Truth Table

For better interpretation of the variables used, refer to Figure 2.

cs2	cs1	cs0	Function	X_i	S_2	S_1	Y_i	S_3	en_3	C_{in}
0	0	0	Add	A_i	0	0	B_i	0	0	0
0	0	1	Sub	A_i	0	0	$\overline{B_i}$	1	0	1
0	1	0	AND	$A_i \wedge B_i$	0	1	0	X	1	0
0	1	1	Sub	A_i	0	0	$\overline{B_i}$	1	0	1
1	0	0	XOR	$A_i \oplus B_i$	1	0	0	X	1	0
1	0	1	Complement A	$\overline{A_i}$	1	1	0	X	1	0
1	1	0	NEG A	$\overline{A_i}$	1	1	0	X	1	1
1	1	1	Complement A	$\overline{A_i}$	1	1	0	X	1	0

Table 2: Truth Table for Intermediate I/0

5 Block Diagram

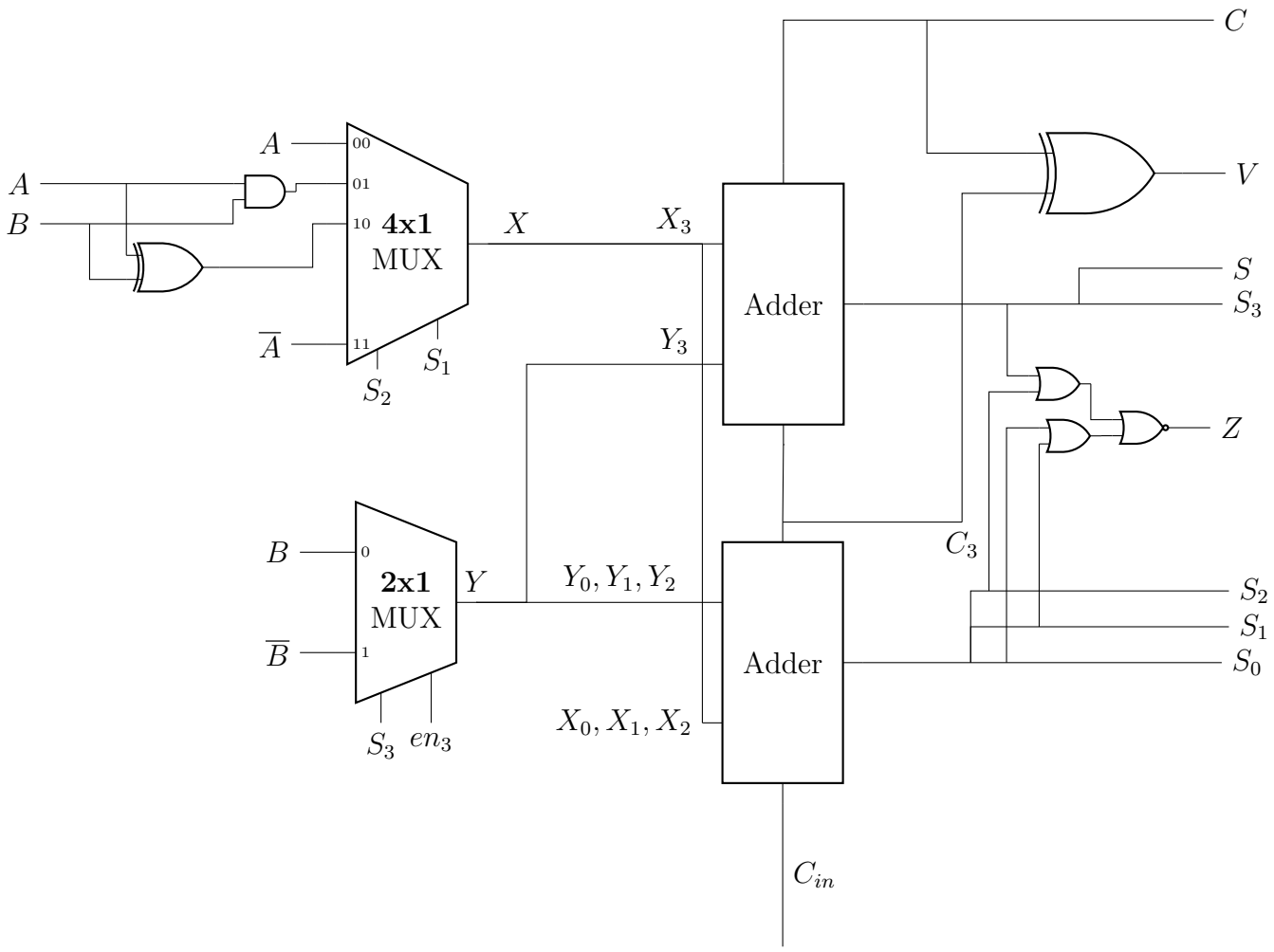


Figure 2: Block Diagram of ALU

6 Complete Circuit Diagram

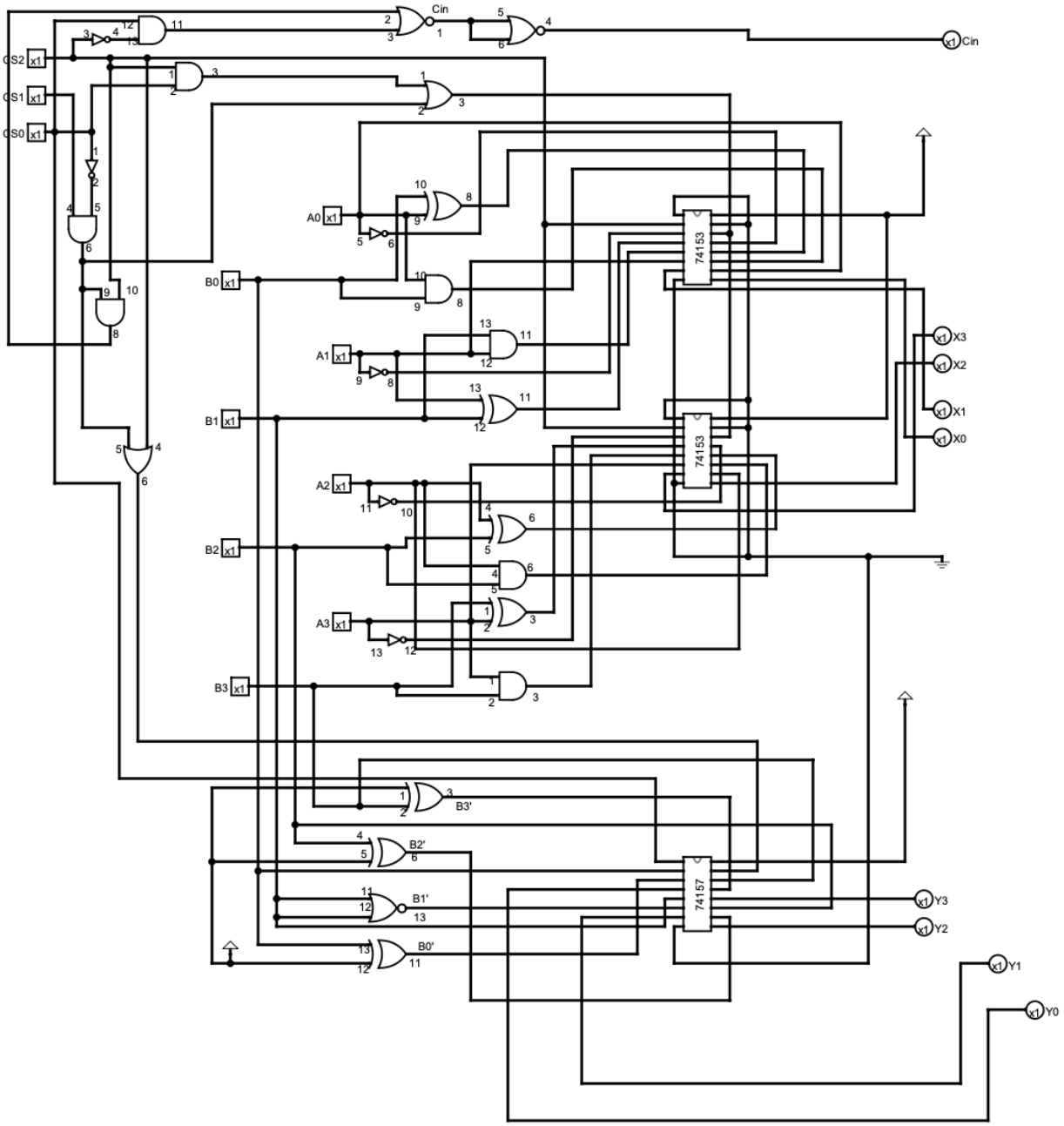


Figure 3: X_i and Y_i circuit

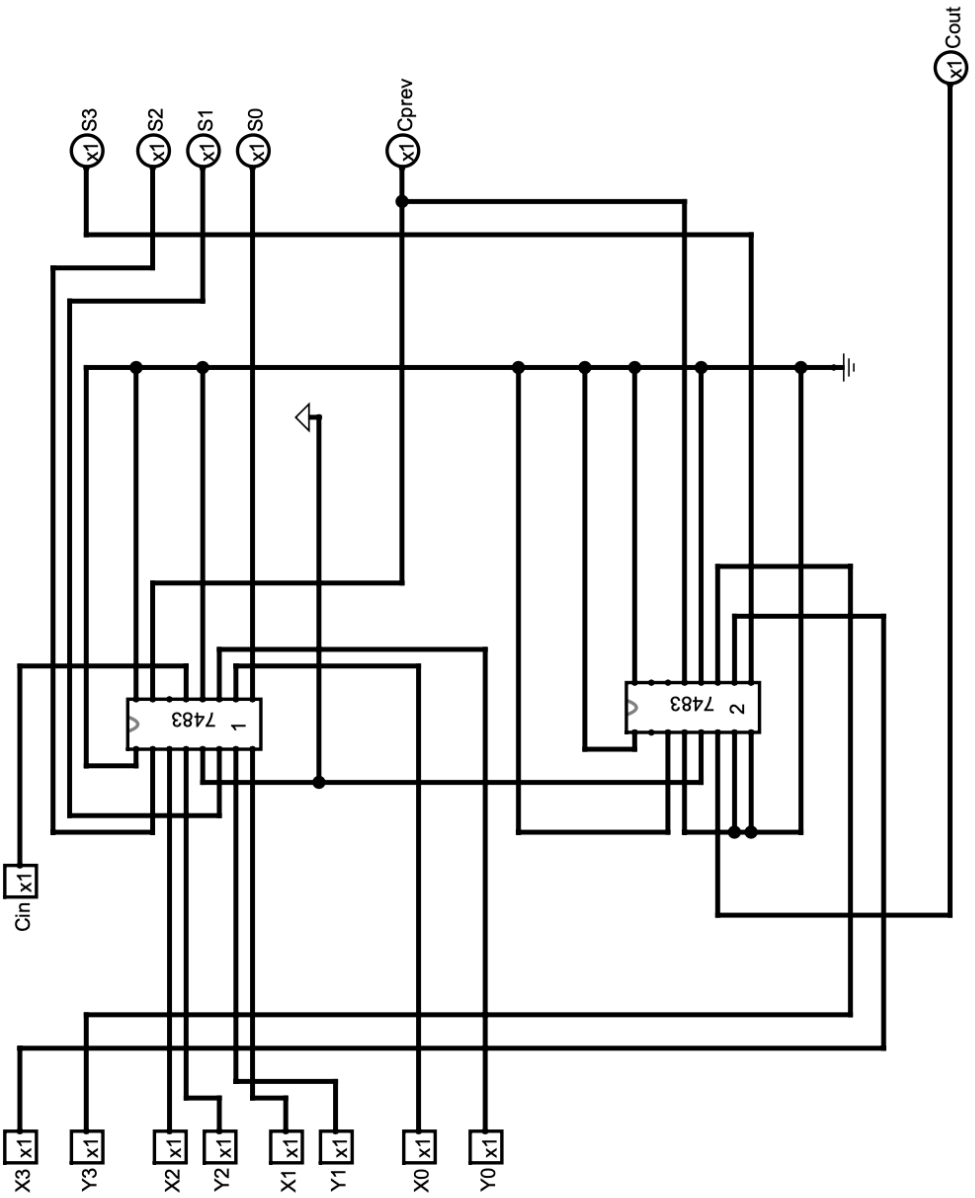


Figure 4: Modified Two Adder ALU System

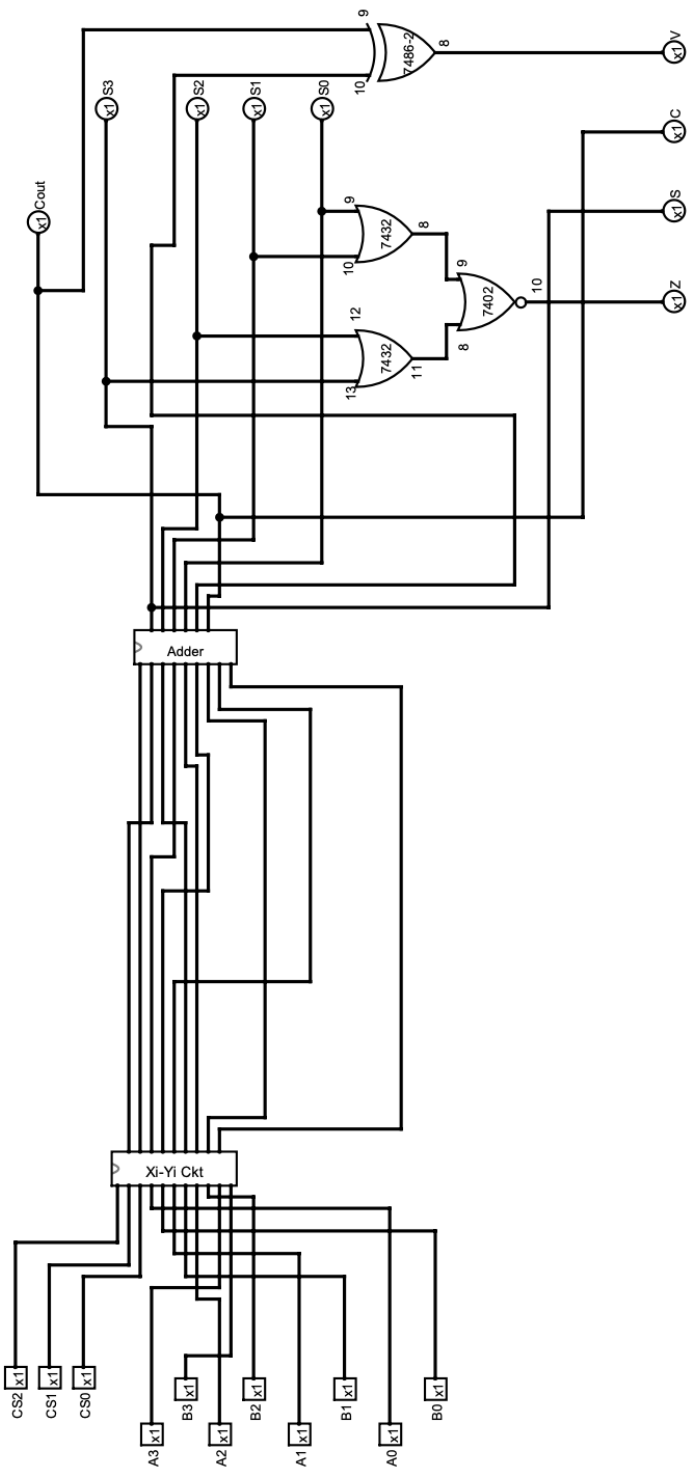


Figure 5: Arithmetic and Logical Unit

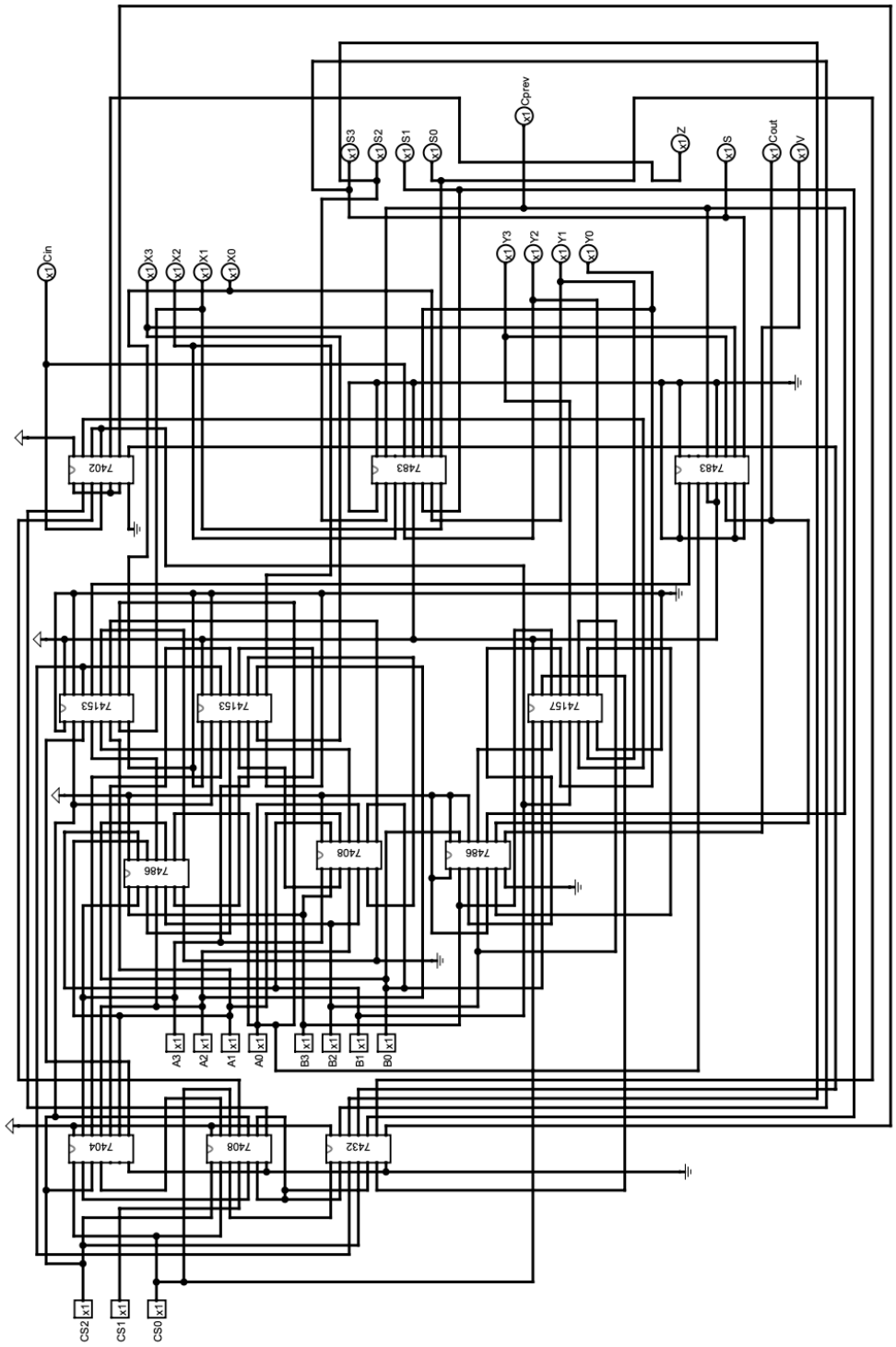


Figure 6: The Complete Design

7 ICs Used with Count as a Chart

IC	Quantity
IC 7402	1
IC 7404	1
IC 7408	2
IC 7432	1
IC 7483	2
IC 7486	2
IC 74153	2
IC 74157	1
Total	12

Table 3: ICs Used with Quantity

8 The Simulator Used along with the Version Number

Logisim - 2.7.1

9 Discussion

In this assignment, we were tasked with implementing a 4-bit ALU capable of performing 3 arithmetic and 3 logical operations.

9.1 Design Finalization

Initially, we had two designs, both requiring a similar number of ICs (13). The key difference was that one design used the adder for the XOR operation while handling the internal carry effectively. However, we ultimately chose the alternative design, which used two adders and a separate IC for the XOR operation. The reason for this choice was that both designs required the same number of ICs, but the latter was simpler to implement and understand. Additionally, in the final design, we had two free XOR terminals on the adder inputs, which we used for extra NOT operations, eliminating the need for an additional IC. We also cross-checked each K-map, noting that the negation of certain functions produced common terms, which could have been leveraged to reduce the number of ICs.

9.2 Optimization

Through extensive analysis, we worked diligently to minimize the number of ICs used. At one point, we realized that we were about to introduce a whole 7404 IC just for a single NOT operation. Instead, we opted to use a 7402 IC (*NOR gate*). Initially, our design required 13 ICs, but we noticed that only two terminals of the 7432 (OR gate) were being utilized. By using two NOR gates, we replicated the OR function, and with another NOR gate, we handled the NOT operation. One actual NOR operation was required for the ZF flag, which would otherwise have consumed an OR and a NOT gate. Using the 7402, we managed to simplify this operation into a single IC. As a result, we removed the 7432 and 7404 ICs, replacing them with a single 7402 IC. Additionally, some NOT operations were handled using unused terminals on the 7486 (XOR gate). Further optimizations were made during debugging, which we discuss later.

9.3 Hardware Implementation

The hardware implementation posed significant challenges, including planning the placement of different components, minimizing wire lengths, and managing bulbs and switches. To maintain a clean hardware layout, we carefully routed wires to minimize crossing and ensure connections were as short as possible. Before powering up the ICs, we checked the voltage on each terminal to avoid damaging the ICs. We also verified the output at each stage before moving on to the next one. As one of the first groups to attempt the hardware implementation, we faced unique challenges. Our initial design failed because we couldn't resolve a bug with one bit of Y_i, which forced us to redesign the ALU from scratch. Despite being more cautious, the second circuit still had several bugs.

9.4 Debugging the ALU

The new ALU design had a bug in the X_1 input. One of the NOT gates (terminals 5 and 6) was not producing the correct output. We were attempting to obtain $\overline{A_1}$ from A_1, but for some reason, it was malfunctioning, lighting up even when A_1 was *ON*. Changing the IC didn't fix the issue, and it was late at night, so we couldn't go out to get a replacement NOT gate. We remembered, however, that we had some unused XOR gates in the second adder (IC 7483). We then managed to obtain $\overline{A_1}$ using the S_3 terminal of the second adder.

On the first day, we also encountered numerous issues with bulbs and switches. The switches didn't fit properly in the breadboard, and we had to modify the pins. Eventually, we switched to a different type of switch that fit without issues. Furthermore, several bulbs were wasted because we didn't initially realize that they had specific voltage and current thresholds.

9.5 Conclusion

Finally, after hours of hard work and troubleshooting, the ALU worked perfectly. The sense of accomplishment was surreal, and we successfully implemented the 4-bit Arithmetic and Logic Unit (ALU) with the minimum number of ICs.

10 Contributions

10.1 Software Implementation:

- **Design:** 2105047, 2405049, 2105046
- **Optimization:** 2105046
- **Logisim Simulation:** 2105047, 2105049
- **Verifying:** 2105055, 2105043

10.2 Hardware Implementation:

- **Power and Ground:** 2105043, 2105049, 2105055
- **Pin Number Tracking:** 2105043, 2105047
- **Wire Management:** 2105043, 2105049
- **Wire Placement:** 2105046, 2105049, 2105055
- **Input:** 2105046, 2105049, 2105055
- **Verification:** 2105043, 2105047, 2105046, 2105055
- **Debugging:** 2105043, 2105046, 2105047, 2105049, 2105055

10.3 Report:

- **Report Documentation:** 2105046
- **Complete Circuit Diagram:** 2105047
- **Block Diagram:** 2105055